

```
verifier et commentaire. from flask_sqlalchemy import
SQLAlchemy
from sqlalchemy.orm import DeclarativeBase
```

```
class Base(DeclarativeBase):
    pass
```

```
db = SQLAlchemy(model_class=Base)
```

Afficher plus ▾

Ton code contient un problème important : la fonction `from_body()` du participant est **en dehors d'une classe**. Elle devrait être intégrée à une classe `Participants`. De plus, quelques commentaires et validations peuvent être améliorés.

Voici une version corrigée et commentée.

```
# Import de SQLAlchemy pour la gestion de la base de données
from flask_sqlalchemy import SQLAlchemy

# Base déclarative SQLAlchemy 2.x
from sqlalchemy.orm import DeclarativeBase

# Classe de base utilisée par tous les modèles
class Base(DeclarativeBase):
    pass

# Initialisation de SQLAlchemy avec la nouvelle base déclarative
db = SQLAlchemy(model_class=Base)

# =====
# Modèle Courses
# =====
class Courses(db.Model):
    __tablename__ = "courses"
```

```
# Clé primaire
id = db.Column(db.Integer, primary_key=True)
```

```
# Nom de la course
```

```
# Distance en kilomètres
distance = db.Column(db.Integer, nullable=False)
```

```
def to_dict(self):
    """
    Convertit l'objet Course en dictionnaire.
    Utilisé pour les réponses JSON.
    """
    return {
        "id": self.id,
        "nom": self.nom,
        "distance": self.distance
    }

def from_body(self, body):
    """
    Remplit l'objet Course à partir des données reçues.
    Les validations peuvent être effectuées ici ou dans utils.py.
    """
    self.nom = body["nom"]
    self.distance = body["distance"]
```

```
# =====
# Modèle Participants
# =====

class Participants(db.Model):
    __tablename__ = "participants"

    # Clé primaire
    id = db.Column(db.Integer, primary_key=True)

    # Informations du participant
    firstname = db.Column(db.String(100), nullable=False)
    lastname = db.Column(db.String(100), nullable=False)
    age = db.Column(db.Integer, nullable=False)

    def to_dict(self):
        """
        Convertit l'objet Participant en dictionnaire.
        """
        return {
            "id": self.id,
            "firstname": self.firstname,
            "lastname": self.lastname,
            "age": self.age,
```

≡ ChatGPT ▾ ✦ Mettre à niveau



Exécuter



```
}

def from_body(self, body):
    """
    Valide les données reçues et remplit l'objet Participant.
    Lève une ValueError si une validation échoue.
    """

    # Vérification des champs obligatoires
    if (
        "firstname" not in body
        or "lastname" not in body
        or "age" not in body
    ):
        raise ValueError("MISSING_FIELDS")

    firstname = body["firstname"]
    lastname = body["lastname"]
    age = body["age"]

    # Vérification du prénom
    if not isinstance(firstname, str) or firstname.strip() == "":
        raise ValueError("INVALID_PARTICIPANT_FIRSTNAME")

    # Vérification du nom
    if not isinstance(lastname, str) or lastname.strip() == "":
        raise ValueError("INVALID_PARTICIPANT_LASTNAME")

    # Vérification de l'âge
    if not isinstance(age, int) or age < 18 or age > 120:
        raise ValueError("INVALID_PARTICIPANT_AGE")

    # Enregistrement des valeurs nettoyées
    self.firstname = firstname.strip()
    self.lastname = lastname.strip()
    self.age = age
```

Corrections apportées

-  Ajout de la classe Participants .
-  Déplacement de from_body() dans la classe Participants .
-  Ajout de commentaires explicatifs.
-  Ajout de to_dict() pour le modèle Participants .
-  Mise en forme conforme à PEP 8.
-  Validation de l'âge (18 à 120 ans).
-  Nettoyage des chaînes avec strip() .

Si tes participants doivent être associés à une course (ce qui est généralement le cas dans ton projet), il faudra aussi ajouter une **clé étrangère** (course_id) et une **relation**

SQLAlchemy (relationship) entre Courses et Participants .



Cette conversation vous aide-t-elle jusqu'ici ?

